CSC3041F 2026
Class Test

Automated Planning and Control

Department of Computer Science
University of Cape Town, South Africa

Notes

1. Answer all questions
2. Test Time: 18h00-18h45, Friday, 6th of March, 2026
3. Test Duration: 45 minutes
4. There are four (4) questions, for a total of 40 marks
5. The **bold** type faced numbers in parentheses indicate the number of points for (sub)questions.

Read through the entire test paper before you start writing so that you can answer the easy questions first. The relative importance of the questions is indicated by the marks that are allocated to them.

Section 1 - Search

Question 1 [9 marks]

1. State whether the following statement is True/False and support your answer: **[2]**

   Greedy best-first search is an uninformed search algorithm

   False [1]. It uses a heuristic for the evaluation (i.e., $f(n) = h(n)$) [1]

2. What is the difference between a problem space and a problem instance? **[2]**

   The problem instance extends the problem space (State space, Actions, and Transition model) by adding initial state [1] and goal state [1]. It may also add a cost function if every action does not have the same cost.

3. For each of the following search algorithms, identify whether a stack, a queue, or a priority queue is used to manage the frontier/openlist: **[3]**

   (a) A* search

   (b) Breadth-first search

   (c) Depth-first search

   Priority Queue, Queue, Stack

4. Under what conditions is it appropriate to make use of Monte Carlo Tree search over alpha-beta search? **[2]**

   When the branching factor is high [1] because even when there is perfect move ordering, the branching factor can only be reduced to $\sqrt{b}$ for alpha-beta search. When you cannot create a reliable heuristic [1] (e.g., Incredibly difficult to create a reliable heuristic for Go unlike Chess).

Question 2 [9 marks]

Consider Algorithm 1 and answer the questions that follow:

1. Describe the purpose of each of the lines labelled 1-4 in Algorithm 1. **[4]**

   Line 1 = select a node to explore through either exploration or exploitation [1 mark]. Line 2 = Generates the child nodes based on possible actions [1 mark]. Line 3 = Runs simulation(s) of the game [1 mark]. Line 4 = Propagates the number wins and visits up the tree [1 mark].

Algorithm 1 Monte-Carlo Tree Search

```
function MONTE-CARLO-TREE-SEARCH(state)
    tree ← Node(state)
    while Is-Time-Remaining() do
        leaf ← Select(tree)                          ▷ Line 1
        child ← Expand(leaf)                         ▷ Line 2
        result ← Simulate(child)                     ▷ Line 3
        Back-Propagate(result, child)                ▷ Line 4
    end while
    return the move in Actions(state) whose node has highest number of playouts
end function
```

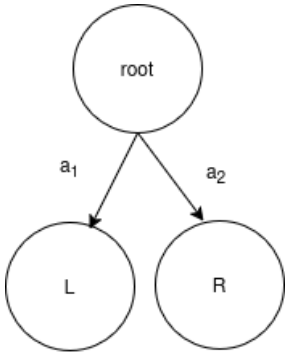


Figure 1: Truncated game tree for Monte-Carlo Tree Search

2. Consider the truncated game tree shown in Figure 1. Which action between a_1 and a_2 will be chosen by the algorithm when using UCB1 (i.e., $\frac{U(n)}{N(n)} + C \times \sqrt{\frac{logN(Parent(n))}{N(n)}}$) with C = 2, and simulation returns the value 5:10 for the node labelled L, and 6:10 for the node labelled R. Demonstrate all the steps to that lead to the answer. **[5]**

   First backpropagation means Root will have 5/10. Second backpropagation means Root will have 11/20 [1 mark]. L UCB1 calculation is 5/10 + 2*sqroot(log (20) / 10) [1 mark]. R UCB1 calculation is 6/10 + 2*sqroot(log (20) / 10) [1 mark]. Both actions have same number of simulations so any action can be returned [2 mark]

Question 3 [12 marks]

Consider the problem illustrated in Figure 2 where the world is a 3×4 gridworld. The car can only move to cells that are adjacent to the cell currently occupied by the car at any time step. The cost of moving between adjacent cells is uniform and can be set to

1. In this problem, we do not allow wrap-around movement. As such, if you are at the bottom-right cell (i.e, (2,3)), you cannot move down or right.

Note: accept traditional 4×3 indexing for student solutions. Allow student indexing to start from other values.

1. Specify a formal problem instance to capture the task if it were to be solved via a search algorithm **[6]**

   State space = {(0,0), ... } [1 mark]. Actions = { Up, Down, Left, Right } [1 mark] (Note: some students may allow diagonal movement). State transition function is T((a,b), action) = (a-1, y) if action = U and $a > 0$, etc. [1 mark] (note: may be slightly different depending on chosen actions and indexing of positions). C(s,a,s')=1 [1 mark]. Init = (2, 3) [1 mark]. Goal = {(0,0), (0, 2)} [1 mark]

2. Specify a reliable heuristic function via problem relaxation to allow an informed algorithm to solve this problem **[1]**

   If U, L, R, D are chosen as actions, then metrics like Manhattan will suffice [1 mark]. (Note: If diagonal movement is allowed, then no metric that overestimates distance should be allowed)

3. Provide evidence to demonstrate that the created heuristic function is admissible when the car is in its initial position **[1]**

   The closest parking spot is at (0, 2) and the init. position is at (2, 3) (note: depends on the chosen indexing), the solution must demonstrate that we will have $h(init) \leq h^*(init)$ [1 mark]

4. Demonstrate how the car identifies the nearest parking spot using the A* search algorithm, guided by the proposed heuristic. **[4]**

   Initial creation of the frontier + closed list [1 mark]. Update to frontier + closed list [1 mark] and node expansion [1 mark]. There may be another expansion, depending on the actions chosen. Finding goal [1 mark]

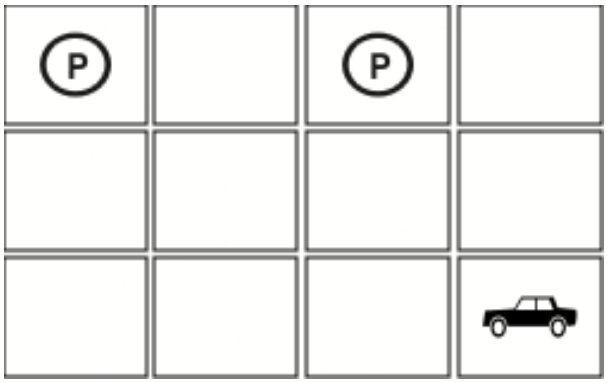


Figure 2: Illustration of a game where a car needs to find the closest parking sport

Section 2 - Reinforcement learning

Question 4 [10 marks]

1. An agent interacts with a finite Markov Decision Process (MDP) indefinitely. It follows an ϵ-greedy policy, where ϵ_t decays over time according to a schedule such that $\epsilon_t > 0 \; \forall \; t$, and $\sum_{t=1}^{\infty} \epsilon_t = \infty$. Which of the following statements is true? **[1]**

   (a) The agent's policy becomes fully greedy in finite time
   (b) The value function V^{π} converges to V for any learning rate.
   (c) Exploration will eventually stop completely.
   (d) Every state–action pair will be visited infinitely often.
   (e) The agent always chooses the optimal action once $\epsilon_t < 0.01$

   d

2. Which of the following statements is true about a stochastic policy? **[1]**

   (a) A stochastic policy can sometimes achieve higher expected return than the best deterministic policy in stochastic environments.
   (b) Deterministic policies are not allowed in MDPs.
   (c) A stochastic policy ignores the MDP transition probabilities when computing expected returns.
   (d) A stochastic policy can assign zero probability to an action from a given state.
   (e) Q-values are undefined for stochastic policies.

   d

3. State which of the following statements about the discount factor γ in $V^{\pi}(s)$ is correct? **[1]**

   (a) $\gamma = 0$ makes $V^{\pi}(s)$ depend only on the immediate reward R(s, a).
   (b) $\gamma = 1$ makes $V^{\pi}(s)$ depend only on future rewards.

(c) $\gamma = -1$ makes all future rewards equally important as immediate rewards.
(d) γ directly changes the transition probabilities of the MDP.
(e) γ determines which actions are permitted in each state.

a

4. Which of the following statements best describes what $Q^\pi(s, a)$ tells us? **[1]**
   (a) The maximum return possible from state s after taking action a, regardless of π.
   (b) The immediate reward $R(s, a)$ received when taking action a in state s under policy π.
   (c) The expected probability, under policy π, that taking action a in state a leads to the highest reward state.
   (d) The expected return starting from state S, taking action a, then following policy π.
   (e) The expected return of taking action a in state s, averaged over all actions in π.

   d

5. Which of the following statements regarding fully and partially observable environments is incorrect? **[1]**
   (a) In a fully observable environment, the current observation is sufficient to represent the state for decision-making purposes.
   (b) In a partially observable environment, different underlying states may correspond to the same observation received by the agent.
   (c) Even in a fully observable environment, transition dynamics may still be stochastic
   (d) In a partially observable environment, the agent may use past observations to form a belief over possible states.
   (e) In a fully observable environment, the optimal policy is always deterministic.

   e

6. An agent is in a single-state MDP (s_1) with two actions, a_1 and a_2, that loop back to the same state, see Table 1. The agent follows a stochastic policy π where $\pi(a_1|s_1) = 0.6$ and $\pi(a_2|s_1) = 0.4$. The discount factor is $\gamma = 0.5$
   (a) Compute $V^\pi(s_1)$ **[3]**
   $V^\pi(s_1) = 0.6 * (3 + 0.5V^\pi(s_1)) + 0.4 * (5 + 0.5V^\pi(s_1))$
   $V^\pi(s_1) = 1.8 + 0.3V^\pi(s_1) + 2 + 0.2V^\pi(s_1) = 3.8 + 0.5V^\pi(s_1)$
   $V^\pi(s_1) = 7.6$
   (b) Compute $Q^\pi(s_1, a_1)$ and $Q^\pi(s_1, a_2)$ **[1]**
   $Q^\pi(s_1, a_1) = R + \gamma V^\pi(s_1) = 3 + 0.5 \times 7.6 = 6.8$ [0.5 marks]
   $Q^\pi(s_1, a_1) = 5 + 0.5 * 7.6 = 8.8$ [0.5 marks]

Action	Next state	Reward
a_1	s_1	3
a_2	s_1	5

Table 1: State transitions and costs

(c) In a state with multiple possible actions, explain why the value of the state under a policy might be lower than the highest possible Q-value of any single action. **[1]**

The state value averages over all actions according to the policy. If the policy sometimes chooses suboptimal actions $V^\pi(s)$ is less than the maximum $Q^\pi(s, a)$ [0.5 must mention averaging over actions. 0.5 must mention policy may choose suboptimal actions]